

HIGH-LEVEL DESIGN

AI in Air-Gapped Environments — Lessons Learned & Best Practices (HLD)

Principles, reference architecture, governance, and risk controls for deploying AI/LLM capability inside a disconnected security boundary, distilled from ION's development

DOCUMENT TYPE

HIGH-LEVEL DESIGN

DATE

2026-07-15

OWNER

ION Maintainer (ixion36)

ION VERSION

v0.51.0

CLASSIFICATION

PUBLIC

AUDIENCE

Design authorities, security leadership, accreditors, architects evaluating AI capability for disconnected estates

Contents

1.1 Purpose

1.2 Operating context

1.3 Summary of the journey

P1 — Inference stays inside the boundary

P2 — AI is advisory; a human owns every consequential action

P3 — Constrain output with fixed contracts, validated server-side

P4 — Treat all model input as adversary-influenced

P5 — Ground in evidence; validate citations mechanically

P6 — Measure quality with a feedback ledger and an offline eval harness

P7 — Every AI feature degrades gracefully to "absent"

P8 — Deterministic first, model second

P9 — Context and budget discipline: nothing silent

P10 — New AI data-at-rest is a governed surface

P11 — The model is part of the supply chain

P12 — Low confidence escalates to a human, automatically

P13 — Observability without egress

1. Introduction

1.1 Purpose

This document captures the lessons learned and resulting best practices from developing, deploying, and operating AI/LLM capability inside **ION**, a security-operations portal built for air-gapped and siloed environments. It is written at the high-level-design altitude: principles, reference architecture, governance, and risk — the material a design authority needs to decide *whether and how* to admit AI capability into a disconnected security boundary.

Every practice in this document was earned in production: each principle cites the concrete ION incident, defect, or design decision that produced it. The companion **LLD** ([docs/AI_AIRGAP_BEST_PRACTICES_LLD.md](#)) descends to implementation level — algorithms, configuration surfaces, code patterns, and test strategy — for engineering teams building to these principles.

The document is deliberately customer-agnostic and contains no compliance-regime-specific material; it composes with whatever accreditation overlay the deploying organisation requires.

1.2 Operating context

The practices assume an environment with these properties, all true of ION's target estates:

- **No external connectivity.** No cloud AI APIs, no telemetry egress, no model-vendor callbacks. Everything the AI needs — weights, embeddings, retrieval corpus — lives inside the boundary.
- **Adversary-influenced input by design.** The AI's primary input is security telemetry (alerts, packet captures, log content). An adversary who triggers an alert *authors part of the model's input*.
- **Single-tenant, authenticated user base** of security analysts; the AI assists but does not replace their judgement.
- **Accreditation pressure.** Every new data store, egress path, or autonomous behaviour must be explainable to a design authority.

1.3 Summary of the journey

ION's AI capability ("Bob") grew over ~50 releases from a single chat endpoint to: per-alert-type investigation guidance, retrieval-augmented grounding over four context sources, evidence-cited auto-investigation, AI-drafted case-closure notes, feedback-measured verdict suggestion with per-rule confidence thresholds, and an offline evaluation harness. Roughly half the practices below exist because something *failed silently* first. The single most repeated lesson: **with LLMs, the dangerous failures are quiet** — a truncated prompt, a defaulted context window, an eval reading the wrong key — and only measurement and contract-pinning make them loud.

2. Principles

Each principle states the practice, then the ION lesson that produced it.

P1 — Inference stays inside the boundary

All model inference runs on infrastructure inside the security boundary (ION: a local Ollama host co-deployed with the application). Model weights are provisioned as versioned artefacts through the same offline supply channel as the application image; ION side-loads GGUF weight files for estates that cannot pull from a registry. There is no fallback path to an external API — a missing model host degrades the feature, never re-routes the data.

Lesson: designing for "no fallback" from day one is far cheaper than retrofitting it. Every AI feature in ION was written against a local endpoint that might be absent, which forced graceful degradation (P7) as a side effect.

P2 — AI is advisory; a human owns every consequential action

The model never closes a case, blocks an IP, or mutates state on its own authority. AI verdicts are suggestions rendered alongside the evidence; the analyst closes. Where the AI drafts text (closure notes, summaries), the analyst reviews and may revert before anything is committed. Autonomous *collection* is acceptable (gathering evidence, enriching indicators); autonomous *action* is not.

Lesson: this is the control that makes every other imperfection survivable. Hallucination, prompt injection, and model regression all have bounded blast radius when the output is a suggestion a trained analyst must ratify.

P3 — Constrain output with fixed contracts, validated server-side

Free-text model output is a liability; structured output pinned to server-owned schemas is an asset. ION pins verdict outputs to a fixed JSON envelope whose verdict field must be a member of a server-side enum, validates the envelope after every call, and treats parse failure as "inconclusive" — never as data. The contract text is appended to every prompt *last* and is exempt from all truncation and budgeting logic (P9).

Lesson (incident): an overlong prompt once cost Bob its output contract — the local runtime truncated prompts *from the front*, where the contract lived, and the model silently reverted to free text. The fix (token budgeting with the contract never droppable) is now structural.

P4 — Treat all model input as adversary-influenced

Alert names, log fields, packet payloads, and file contents are attacker-authored text that will be interpolated into prompts. ION wraps all telemetry in a delimited input block, sanitises values, and

instructs the model that the block is data, not instructions. Nothing that arrives from a monitored system is ever concatenated into the instruction section of a prompt. The same discipline applies to retrieval: RAG corpus text is first-party operator content (KB articles, playbooks, closed cases) — *not* raw telemetry — so retrieval cannot become an injection amplifier.

Lesson: prompt injection defence in a SOC context is not hypothetical — the adversary literally chooses the strings in the alert. Input wrapping plus a fixed output contract plus advisory-only output (P2) is the layered answer; no single layer is sufficient.

P5 — Ground in evidence; validate citations mechanically

A model asked to reason without evidence will confabulate from whatever context it has. Two ION incidents shaped this principle:

- The AI closure-note rewriter, given no case evidence, parroted its only concrete context — a similar-case precedent — producing notes like "a similar case was closed as benign" with no case-specific rationale. The fix: feed the case's own bounded evidence block, structure the note around quoted evidence, and demote precedent to at most one trailing sentence.
- The auto-investigation feature requires every model finding to cite evidence-ledger entry IDs that the *server* assembled. Citations are validated mechanically after the call: invalid references are dropped, findings with no surviving citations are discarded, and a decisive verdict left unsupported is downgraded to inconclusive.

Best practice: the server gathers the evidence, the model must cite it, and the server verifies the citations. Hallucination is bounded by construction, not by prompt-pleading.

P6 — Measure quality with a feedback ledger and an offline eval harness

Every AI verdict is recorded against the eventual human decision in an append-style feedback ledger, giving per-rule agreement rates over time. An offline evaluation harness replays stored prompts against candidate models/configurations and scores precision/recall/F1 before anything changes in production. Confidence thresholds (P12) are tuned from this data, not intuition.

Lesson (incident): ION's eval harness read the model reply from the wrong response key for several releases — every sample scored as an abstention and the metrics were null. Nobody noticed quickly because *an eval that produces no signal looks like an eval that found no problems*. Treat the evaluation pipeline itself as production code: test it, and alarm on degenerate outputs (all-abstain, all-agree).

P7 — Every AI feature degrades gracefully to "absent"

The model host being down, the embedding model missing, or the vector store unavailable must produce a clean no-op — a hidden panel, an empty context section, a 503 with a clear message — never a crash, a

queue backup, or a data-loss path. ION's retrieval layers return empty lists on any failure; its background embedding loops skip-and-retry; its UI features hide when the AI is unreachable.

Lesson: this is what made **default-on** safe. ION's RAG layers ship enabled because a site without an LLM host pays only a cheap no-op; a site with one gets the capability with zero configuration. In an air-gapped product you cannot assume a working model host, so degradation discipline is not optional polish — it is the deployment model.

P8 — Deterministic first, model second

Where a structured mechanism can answer, use it before the statistical one. ION's playbook-recommendation layer matches on operator-authored trigger conditions (rule patterns, ATT&CK techniques) first — precise, auditable, and functional with no model at all — and falls back to embedding similarity only when nothing matches structurally. Its prompt-template selection is a deterministic five-tier matcher, not a classifier.

Lesson: deterministic paths are explainable to accreditors, testable with exact assertions, and free. Reserve the model for the part of the problem that is genuinely fuzzy.

P9 — Context and budget discipline: nothing silent

LLM runtimes fail quietly at their limits: context windows default low, truncation eats from one end without error, and embedding models drop input tails. ION sets the context window explicitly (never trusting runtime defaults), maintains an explicit token budget for prompt assembly with a priority order for optional context layers (drop lowest first, log every drop), and clips every retrieval input per-section so no single verbose field can evict the others.

Lessons (three separate incidents): a silently-defaulted context window front-truncated prompts for multiple releases; an unbounded field pushed later sections out of embedding vectors; and long documents lost their tails to the embedding window until chunk-level embedding replaced whole-document vectors. In all three, nothing errored — the quality just quietly sagged. **Assume every limit is silent and make it loud yourself.**

P10 — New AI data-at-rest is a governed surface

AI features accrete state: prompt snapshots, raw model responses, model reasoning text, vector embeddings, feedback ledgers. Each is a new data-at-rest surface that must be capped (ION hard-caps stored prompts/responses), gated where sensitive (reasoning-text storage has an opt-out that also suppresses previously stored rows at the response layer), documented in the security assessment as net-new surface, and included in retention thinking. Embeddings deserve explicit note: they are derived data, regenerable at will, so they can be dropped/rebuilt freely in migrations — ION exploits this to keep vector-schema changes trivial.

P11 — The model is part of the supply chain

Weights, embedding models, and AI-adjacent Python dependencies enter the boundary through the same channel as everything else and deserve the same controls: version pinning, checksum/provenance verification, SBOM inclusion, and vulnerability/malware scanning of the packaging pipeline. Model *selection* is also a supply-chain decision: ION chose a security-domain-tuned open model after comparative evaluation, and **rejected** its chain-of-thought "reasoning" variant for free-text surfaces because reasoning leakage into analyst-visible fields was an unacceptable output-handling risk.

Lesson (incident): ION's dependency-audit gate caught a malicious release of a core web framework (typosquat dependency injected upstream). The same posture must cover model artefacts — a poisoned GGUF is harder to detect than a poisoned wheel, so provenance matters more, not less.

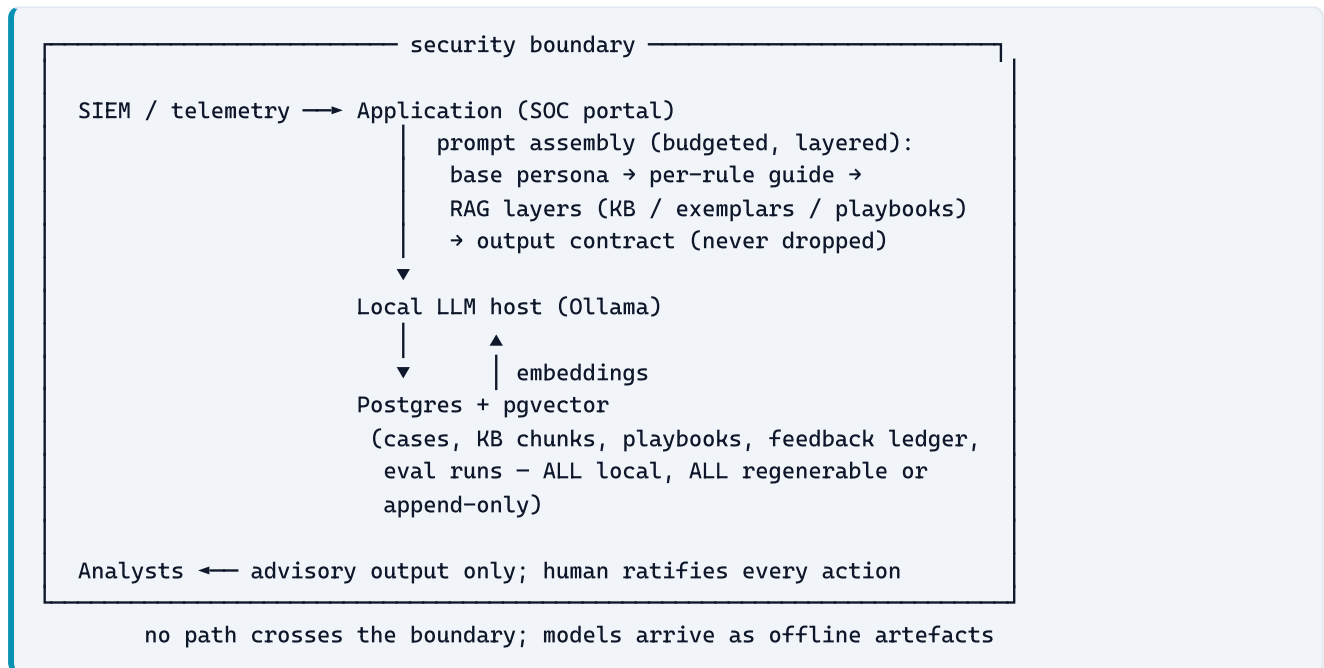
P12 — Low confidence escalates to a human, automatically

AI verdict suggestions carry a numeric confidence; per-alert-type thresholds decide whether the suggestion is surfaced or replaced by an explicit "auto-escalated to analyst" state. A circuit-breaker treats sustained low-confidence output as a signal to stand the AI down rather than let it guess. Thresholds are per-rule (one global threshold fits nothing) and tuned from ledger data (P6).

P13 — Observability without egress

Operational monitoring of the AI (latency, token counts, failure rates, per-loop health) is first-party and opt-in, exported only to operator-controlled collectors inside the estate. Default is off; enabling it is a deliberate deployment decision documented in the estate's data-protection record. Never accept an AI component whose telemetry cannot be pointed exclusively at infrastructure you own.

3. Reference architecture



Key structural choices:

- **One local inference host** serves chat, investigation, and embeddings — one thing to size, monitor, and stand down.
- **Vector store in the primary database** (pgvector) rather than a separate vector service — one fewer container, one backup story, and SQL-joinable retrieval.
- **Background embedding producers** run as advisory-lock-guarded loops (exactly one worker embeds, across N app processes), decoupling embedding cost from request latency.
- **Retrieval corpus is first-party content only** (P4): knowledge-base articles, operator playbooks, human-closed cases.

4. Governance model

1. **Per-release security delta.** Every release records its net-new AI surface (new store, new prompt path, new gate) and a net-new findings count in the standing security assessment. "Internal prompt logic only — no new surface" is itself a recordable claim.
2. **Default posture reviews.** Flipping an AI gate default (off→on) is a governed event justified by the degradation argument (P7) and documented as such.
3. **Evaluation before behaviour change.** Model swaps, threshold changes, and prompt-strategy changes run through the offline eval harness against replayed production prompts; retrieval changes are benchmarked on the real corpus (ION measured its embedding-prefix change at +57% nDCG@10 on the production knowledge base before shipping).
4. **Human-in-the-loop is architectural, not policy.** The absence of state-mutating AI codepaths is verifiable in code review; policy documents alone do not bound an LLM.

5. Risk view (OWASP LLM Top 10 mapping)

RISK (OWASP LLM TOP 10)	POSTURE FROM THESE PRACTICES
LLM01 Prompt injection	Delimited input wrapping + instruction/data separation (P4); first-party-only RAG corpus; advisory-only output (P2) bounds impact
LLM02 Insecure output handling	Fixed JSON contract + server-side enum validation (P3); all rendered output HTML-escaped; chain-of-thought variants rejected for free-text surfaces (P11)
LLM03 Training-data poisoning	No in-boundary training/fine-tuning; model artefact provenance (P11)
LLM04 Model DoS	Token budgets + explicit context windows (P9); timeouts + circuit breakers (P7, P12); background loops batched and lock-guarded
LLM05 Supply chain	Offline artefact channel, pinning, SBOM, audit gate — proven live by a caught malicious dependency (P11)
LLM06 Sensitive info disclosure	No egress (P1); AI data-at-rest capped/gated/documented (P10); observability opt-in and first-party (P13)
LLM07 Insecure plugin design	No autonomous tool execution; deterministic-first design (P8); AI tool surface (MCP) permission-gated per tool and off by default
LLM08 Excessive agency	Advisory-only (P2); collection-not-action rule; low-confidence auto-escalation (P12)
LLM09 Overreliance	Feedback ledger + agreement metrics make analyst/AI disagreement visible (P6); confidence surfaced with every suggestion
LLM10 Model theft	Weights inside the boundary under platform access controls; no model-serving endpoint exposed beyond the application network

6. Lessons-learned register (summary)

#	WHAT HAPPENED	WHAT NOW EXISTS
L1	Runtime front-truncated prompts; the output contract fell off silently	Token-budget guard; contract appended last, never droppable (P3, P9)
L2	Context window silently defaulted to a fraction of the model's capability	Explicit context-window configuration, never runtime defaults (P9)
L3	Eval harness read the wrong response key — metrics were null for releases	Eval pipeline treated as production code; degenerate-output alarms (P6)
L4	Closure-note AI parroted similar-case precedent, lacking any case evidence	Server-gathered evidence blocks; citation-required prompting; precedent demoted (P5)

#	WHAT HAPPENED	WHAT NOW EXISTS
L5	Whole-document embeddings lost article tails to the embedding window	Chunk-level embedding; per-section input clipping (P9)
L6	Retrieval quality unmeasured; embedding change value unknown	Real-corpus retrieval benchmarks gate embedding changes (§4.3)
L7	Malicious release of a core dependency entered the image	Audit gate + pinning; posture extended to model artefacts (P11)
L8	Chain-of-thought model variant leaked reasoning into analyst-facing text	Model-selection review includes output-leakage assessment (P11)
L9	Inconclusive-verdict AI summaries polluted similarity retrieval (boilerplate pulled unrelated cases together)	Decisive-verdict filters on what enters the corpus (P5, P9)
L10	A disabled procedure could have been suggested via stale vectors	Lifecycle rule: deactivation evicts from the vector store (P8, P10)

7. Adoption guidance

For an organisation starting where ION started, the practices sequence naturally:

1. **Foundations first:** local inference (P1), advisory-only (P2), output contracts (P3), input wrapping (P4), graceful degradation (P7). These are cheap at the start and expensive to retrofit.
2. **Then measurement:** feedback ledger and eval harness (P6) *before* investing in retrieval sophistication — you cannot improve what you cannot score.
3. **Then grounding and retrieval:** evidence-cited generation (P5), deterministic-first retrieval (P8), budget discipline (P9).
4. **Continuously:** data-at-rest governance (P10), supply chain (P11), escalation tuning (P12), observability (P13).

The companion LLD provides the implementation detail for each step.